LECTURE 5

Inductive Synthesis with Stochastic Search

Back in 2013, a group at Stanford started publishing a series of papers where they showed the potential of using stochastic search techniques to attack complex synthesis problems that were beyond the capabilities of other synthesis methods. The first of these papers was published in ASPLOS 2013 [1] by Eric Schkufza, Rahul Sharma and Alex Aiken, and it demonstrated the use of stochastic search for superoptimization problems, where the goal is to find a sequence of X86 instructions that are close to optimal for a particular task.

The basis for a lot of this work is a stochastic search technique known as Markov Chain Monte Carlo (MCMC). This technique also forms the basis of the search algorithms used by many probabilistic programming languages such as Church. Back in 2009, Persi Diaconis wrote a very nice tutorial on MCMC that provides a good introduction to the area [2]. In this lecture we will cover some of the highlights, starting with some notation for Markov Chains, before moving into it's application in program synthesis. Towards the end of the lecture, we will also briefly introduce a related technique called Sequential Monte Carlo (SMC). This has not seen much use in inductive syntheis per se, but it has recently become relevant in the context of large language models. We will see more of this in Unit 2.

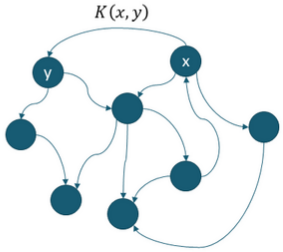
5.1 Markov Chains

Markov Chains

Let χ be a finite set

A Markov chain is defined by a matrix $K(x,y)$: $\chi \times \chi \to \mathbb{R}$

- $K(x,y) \geq 0$
- $\sum_y K(x,y) = 1$
- Probability of transitioning from x to y



Markov Chains

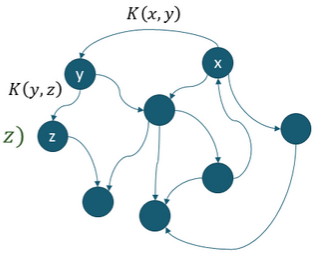
Let χ be a finite set

A Markov chain is defined by a matrix $K(x,y)$: $\chi \times \chi \to \mathbb{R}$

- $K(x,y) \geq 0$
- $\sum_y K(x,y) = 1$

Probability of a series X_0, X_1, X_2 …

- $P(X_1 = y | X_0 = x) = K(x,y)$
- $P(X_1 = y, X_2 = z | X_0 = x) = K(x,y)K(y,z)$



Markov Chains

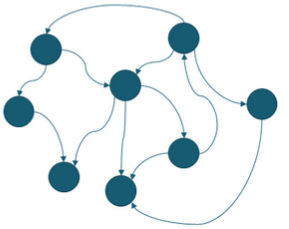
Let χ be a finite set

A Markov chain is defined by a matrix $K(x,y)$: $\chi \times \chi \rightarrow \mathbb{R}$

- $K(x,y) \geq 0$
- $\sum_y K(x,y) = 1$

Probability of a series X_0, X_1, X_2 ...

- $P(X_1 = y | X_0 = x) = K(x,y)$
- $P(X_1 = y, X_2 = z | X_0 = x) = K(x,y)K(y,z)$
- $P(X_2 = z | X_0 = x) = \sum_y K(x,y)K(y,z)$
  - This is matrix multiplication!



A Markov process is a probabilistic process where there is a finite set of states χ, and at each step of the process, the probability of transitioning from a given state x to a different state y is given by a matrix $K(x,y) : \chi \times \chi \rightarrow \mathbb{R}$. Because the values of K are probabilities then it must be the case that $\forall x, y . 0 \leq K(x,y) < 1.0$, and at every state there will be a transition (potentially to the same state), so $\forall x . \sum_{y \in \chi} K(x,y) = 1.0$.

A *Markov Chain* is a sequence of states $x_0, x_1, x_2, ...$ in a Markov process. The probability of the whole chain can be computed as the product of the probability of each transition, so

$$P(x_1 = y | x_0 = x) = K(x,y)$$
$$P(x_2 = z, x_1 = y | x_0 = x) = K(x,y) * K(y,z)$$

Now, if I want to know the probability that $x_2 = z$ given that I started at $x_0 = x$, then I need to consider all the possible states for x_1 that I could have taken to get from x to z. This can be computed as $\sum_{y \in \chi} K(x,y) * K(y,z)$. One of the key observations about Markov chains is that this is actually matrix multiplication. Another way to say this is that K is a matrix that tells me the probability of transitioning from x to y in one step. K^2 is the probability of transitioning in two steps, K^3 is the probability of transitioning in 3 steps, and $K^n(x,y)$ is the probability of transitioning from a state x to a state y in exactly n steps.

5.2 Stationary distributions

Fundamental theorem of (finite) Markov chains

If there is an n_0 s.t. $\forall x, y.\ \ n > n_0 \Rightarrow K^n(x, y) \geq 0$

- i.e. the matrix is connected.
- the matrix must also be aperiodic

$\forall x. \lim_{n \to \infty} K^n(x, y) = \pi(y)$

- The n'th step of a run starting at x has probability close to $\pi(y)$ of being at y if n is large.

We notice that the matrix K and its powers tell us the probability of being in a particular state after some number of steps. Intuitively, we can see that if I start at some state x, then the state y that I end up after one step is heavily determined by x. But if a process has been running for a long time, then my current position will be determined more by the overall structure of the markov process than by where I started. So in a Markov process, we can talk of the *Stationary Distribution* $\pi(x)$ as the probability that a Markov process will be at a particular state in the long run.

The stationary distribution has the interesting property that $\pi(x) = \sum_{x \in \chi} \pi(x) * K(x, y)$. In other words, $\pi = K * \pi$, so the stationary distribution π is an eigenvector of K with eigenvalue 1. The Markov Chain Monte Carlo algorithm (MCMC) is based on a very important property of Markov Chains known as the **Fundamental Theorem of Markov Chains**, which states that if a Markov chain satisfies a few technical requirements, then $\forall_{x \in \chi} \lim_{n->\infty} K^n(x, y) = \pi(y)$. This is a very powerful statement because it tells us that we can compute the stationary distribution by starting at some state and then running the markov process for a long time, and it won't even matter where we started. In order for the theorem to hold, there are a few important technical requirements, the most important one for our purposes is that the markov chain must be fully connected, i.e. it must be possible to reach every state from every other state.

Another important requirement is that the markov process must be aperiodic. The aperiodic property rules out, for example, a markov process that alternates from one state to another, as defined by the matrix $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. Such a process spends all the even steps in one state, and all the odd steps in another state, and the limit in the theorem is not even well defined.

5.3 Search with Metropolis-Hastings

The fundamental theorem of Markov Chains suggests an approach for synthesizing programs. First, let χ be the space of programs. We are going to engineer a $K(x, y)$ such that $\pi(x)$ is high for "good programs" and low for "bad programs". Then we can pick a random start state x_0 and simulate the markov process for n steps for some large n. By the fundamental theorem, the probability that x_n is a good program will be higher than the probability that it is a bad program. The key for this strategy to work, then, is to develop a $K(x, y)$ that has exactly the right properties.

The key step for the above approach to work is to engineer a transition matrix K that has a desired $\pi(x)$ as a stationary distribution. The most popular approach is the Metropolis-Hastings algorithm (MH), first presented in a completely different context by Metropolis, Rosenbluth, Rosenblith, Teller and Teller [3] and later generalized by Wilfred Hastings.

The starting point for the Metropolis-hastings approach is a Markov matrix $J(x, y)$ that is known as a *proposal distribution*. The only formal requirement on the proposal distribution is that $J(x, y) > 0$ iff $J(y, x) > 0$, although in practice the specific form of the proposal distribution can have a big impact on how fast the algorithm converges. Starting with the proposal distribution $J(x, y)$ and the desired stationary distribution $\pi(x)$, the approach defines a quantity $A(x, y) = \frac{\pi(y)*J(y,x)}{\pi(x)*J(x,y)}$ it calls the acceptance ratio, and from that it defines the distribution $K(x, y)$ as follows.

$$K(x,y) = \begin{cases} J(x,y) & \text{if } x \neq y \text{ and } A(x,y) \geq 1 \\ J(x,y) * A(x,y) & \text{if } x \neq y \text{ and } A(x,y) < 1 \\ J(x,y) + \sum_{z:A(x,z)<1} J(x,z)(1 - A(x,z)) & \text{if } x = y \end{cases}$$

The argument for why this $K(x, y)$ will have the desired stationary distribution $\pi(x)$ is relatively straightforward. The key observation is that $\pi(x) * K(x, y) = \pi(y)*K(y, x)$. From that observation it follows that $\sum_x \pi(x)*K(x, y) = \sum_x \pi(y) * K(y, x) = \pi(y) \sum_x K(y, x) = \pi(y)$, so $\pi = K\pi$.

An important advantage of this definition of K is that it is relatively easy to sample from this distribution, assuming we can easily sample from the proposal distribution J. Given a state x, the approach is to take a sample from the distribution $J(x, y)$, and compute the value of $A(x, y)$ by comparing the value of $\pi(x)$ with the value of $\pi(y)$. If $A(x, y) > 1$, which usually means that $\pi(y)$ is better than $\pi(x)$, then we transition to state y. If $A(x, y)$ is worse, then we transition to y with probability $A(x, y)$ and otherwise we stay in x.

Another important feature of the MH approach is that π only appears as a ratio $\pi(y)/\pi(x)$. This means that the algorithm can work even if the function π is not normalized. In fact, most approaches just use a scoring function as opposed to a distribution.

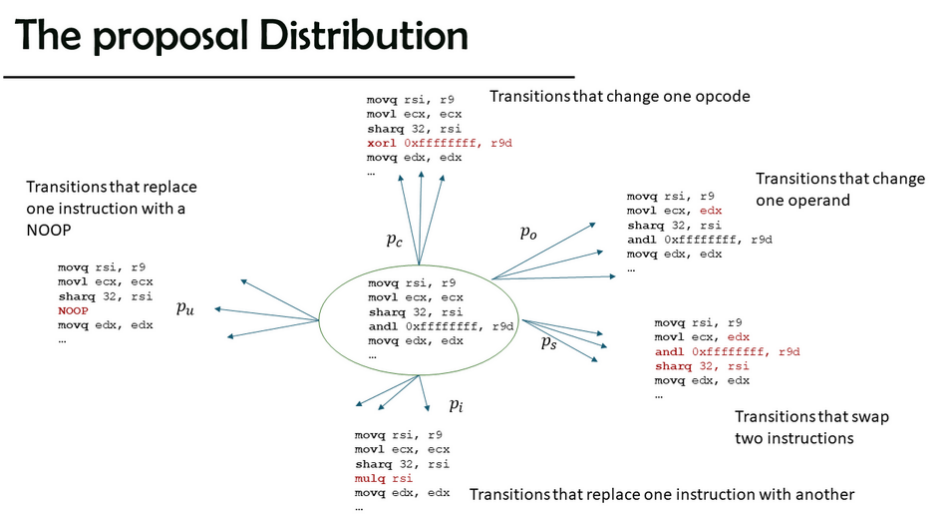
5.4 Metropolis-Hastings for Program Synthesis

Applying the MH approach to program synthesis boils down to defining the program space, defining a desired stationary distribution π and a proposal distribution J. Given a programming by example problem, it is tempting to simply define J as a uniform distribution (so transitioning from any program to any other program has the same probability) and π as having probability 1 for the correct program and zero for every other program. Unfortunately, it's not so simple. For one, the K would not be well defined, since the MH approach would lead to a division by zero (note that the requirement of the fundamental theorem of markov chains implies that $\pi(x)$ cannot be zero for any x). We could make $\pi(x)$ for incorrect programs x be some $\epsilon > 0$, so that at least we satisfy the formal requirements of the algorithm, but even that would not be particularly effective as a search mechanism. Under such a distribution, the search would devolve to randomly generating programs and testing them against the examples; hardly an efficient search strategy.

In order for MH to actually be effective as a search strategy, we need a stationary distribution π that allows us to tell whether a program is getting closer to being correct, and we need a proposal distribution J that gives priority to programs whose behavior is going to be similar to the current program, so that we do not easily lose track of what we had already learned from the current search by just jumping to a completely different program every time (jumping to a completely different program occasionally is good, because it helps the algorithm escape from local minima).

As an example, consider the system of Schkufza, Sharma and Aiken. In that system, the goal is to synthesize a sequence of assembly instructions $\mathcal{R}$ that is equivalent to some reference program $\mathcal{T}$ but more efficient.

The program space: The program space for the paper corresponds to sequences of assembly instructions of a bounded length. The bound on the length is what makes the space finite.



The proposal distribution: Rather than explicitly define the proposal distribution $J(\mathcal{T}, \mathcal{R})$, the paper defines the probabilistic process by which a program trying to simulate the Markov process would transition from a program $\mathcal{T}$ to a program $\mathcal{R}$. This process is defined by five separate probabilities:

- p_c: the probability of swapping the opcode of an instruction in $\mathcal{T}$ for a compatible opcode, one that requires the same number and type of parameters. That means that given a program $\mathcal{T}$, if there are K ways of swapping any of it's opcodes for another legal one, then the actual probability $J(\mathcal{T}, \mathcal{R})$ for another program $\mathcal{R}$ that differs from $\mathcal{T}$ by the opcode of one of its instructions would be p_c/K.
- p_o: the probability of replacing one random operand from one random instruction in $\mathcal{T}$ with an alternative legal operand.
- p_s: the probability of transforming from $\mathcal{T}$ to $\mathcal{R}$ by swapping two instructions.
- p_i: this is the probability of replacing an instruction in $\mathcal{T}$ with a random instruction.
- p_u: the probability of replacing an instruction in $\mathcal{T}$ with a NOOP.

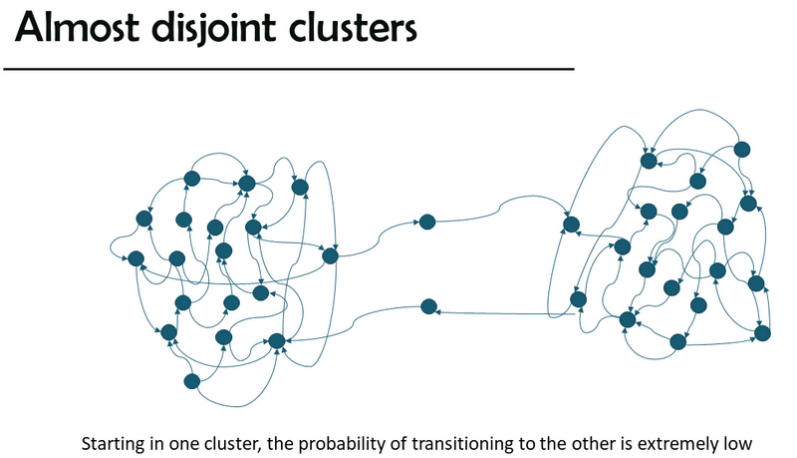
So what that means is that in simulating the Markov process, given a program $\mathcal{T}$, then with probability p_s, for example, you can pick two instructions at random and swap them, or with probability p_u, you can pick one instruction and random and replace it with a NOOP.

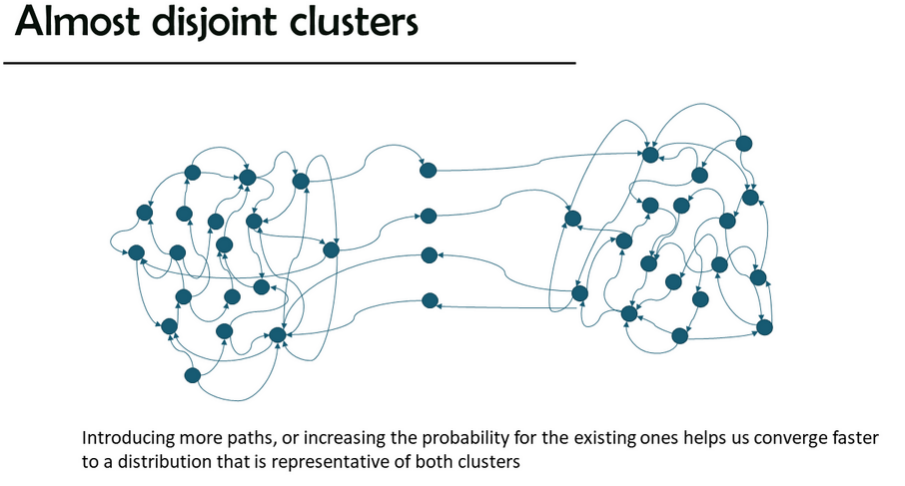
The stationary distribution: In that context, the target stationary distribution $\pi(\mathcal{T})$ has two components, a correctness component $eq(\mathcal{R}, \mathcal{T})$ and a performance component $perf(\mathcal{R}, \mathcal{T})$

$$\pi(\mathcal{T}) = \frac{1}{Z}(exp(-\beta(eq(\mathcal{R},\mathcal{T}) + perf(\mathcal{R},\mathcal{T}))))$$

We can ignore the normalization constant $1/Z$ from now on because as mentioned before, it is not relevant for the algorithm. The constant β is just a tuning parameter.

The correctness component is computing by running the candidate program $\mathcal{R}$ on the test inputs and computing a distance between its output and the output of the original program. Because the paper deals with assembly programs, the output includes the values of all registers, as well as the values of memory accessed by either program, and the distance is just the bitwise Hamming distance. The performance component is computed by evaluating the candidate program through a performance model that assigns a cost to each instruction.





Additional embellishments The program space, the proposal distribution and the stationary distribution can be used together to find programs using MH, although the paper also describes some additional embellishments that help the system converge to efficient programs more efficiently. The most important of them is that the search is conducted in two stages. In the first stage, the performance component is completely ignored, allowing the search to discover correct programs that are very different from the initial one, even if their performance is not great. Then, a set of these correct programs discovered in the first phase are used as starting points for a second phase search that includes the performance term.

The reason for doing this is illustrated in the figure. The idea is that in general, in any Markov process, if you have two tightly connected clusters of states with only a few low probability paths between them, then MH can take a long time to transition from one to another. If, on the other hand, you increase the probability of those paths, or you add more paths, then it will be easier for MH to switch from one cluster to the other. The performance term reduces the probability of any transition that involves a step that reduces the performance of the program. By eliminating the performance term during stage 1, it becomes easier for the algorithm to escape its initial cluster and find programs that are very different from the original program.

5.5 Metropolis-Hastings with ASTs

The MH approach has also been used to search simpler expression trees in an expression language. For example, the survey by Alur et al. [4] which introduced the syntax guided synthesis format (SyGuS) describes a stochastic synthesis approach over ASTs. As before, the approach is sumarized by the space of programs, the proposal distribution and the stationary distribution.

The program space: In this setting, the space of programs corresponds to the space of ASTs up to a size bound N.

The proposal distribution: As in the previous example, the proposal distribution is described in terms of the probabilistic process by which a program trying to simulate the Markov process would transition from a program e to a program e'.

In this process, the algorithm first picks an AST node v uniformly at random from the AST for e. Let e_v be the entire sub-expression rooted at v. The expression at v can then be replaced by a randomly generated expression of the same size as e_v.

The stationary distribution The stationary distribution, is defined in terms of a cost function $C(e)$.

$$\pi(e) = \frac{1}{Z} exp(-\beta C(e))$$

Where the cost $C(e)$ is equal to the number of examples that the function got wrong.

Discussion

This very generic algorithm is quite simple, although it is not very effective, as the comparison with other algorithms done in the survey [4] illustrated. Although the root causes for the poor performance were not explored in the paper, we can suggest a few possibilities. First, the stationary distribution, the scoring function, is not precise enough. Programs that are close to correct but still fail all the examples will get the same low score as completely wrong programs. In the superoptimization paper, by contrast, the fact that a program will potentially touch many different memory locations and registers means that almost correct programs have a higher likelihood of getting at least some bits of the output correct. Another thing we observe in this approach is that the proposal distribution can really only make big changes to the program. If a program is close to correct, and only one node in the middle of the AST has to be changed, then the proposal distribution cannot make this local change without re-randomizing the entire subtree below that AST node.

In general, the MH approach can be a very powerful search method, particularly in cases where we need to search for programs larger than what explicit search techniques, and where we have enough intuition about the space to define good proposal distributions and scoring functions. Carefully defining these two functions is crucial for this method to actually work.

5.6 Sequential Monte Carlo

The MH approach is one way to produce a representative sample from a target distribution π given a markov process K that has π as its stationary distribution.

We are now going to describe a related technique known as Sequential Monte Carlo (SMC), which is also known as particle filtering [5]. To better understand this technique, we will take a little diversion to a very different kind of problem.

Suppose you are on a boat navigating the ocean; your coordinates at time t are given by a variable x_t. One day, your sextant breaks, so within a few days, you only have a vague idea of where you are. Your position is given by a distribution $P_0(x_0)$. With some work, you have managed to improvise a makeshift sextant, but it's not very accurate. When you take a measurement at time t, you get a reading y_t that is related to your actual position x_t by a distribution $P(y_t|x_t)$. Finally, your knowledge of the currents in your region tell you that your position at time $t+1$ is related to your position at time t by a distribution $P(x_{t+1}|x_t)$.

If you accumulate a series of measurements $y_1, y_2, ..., y_t$, you can use the sequential monte carlo approach to estimate your position x_t given the

measurements. To understand how this works, let's start with a simple approach. We start by taking a set of samples $X^{(0)} = \{x_1^{(0)}, x_2^{(0)}, ..., x_N^{(0)}\}$ from the initial distribution $P_0(x_0)$. We will call each of these a particle. Then, at each step t, we simulate the next step by sampling a new set of samples $X^{(t+1)} = \{x_1^{(t+1)}, x_2^{(t+1)}, ..., x_N^{(t+1)}\}$ from the conditional distribution $P(x_{t+1}|x_t)$. So now, for each initial particle $x_i^{(0)}$, we have a whole sequence of corresponding particles $x_i^{(t+1)}$ that represents a possible trajectory of the boat. If we want to estimate the probability that this trajectory corresponds to the true trajectory of the boat, we can compute the probability of the measurements given this trajectory as $\prod_{t=1}^{T} P(x_i^{(t)}|y_t) = \prod_{t=1}^{T} \frac{P(y_t|x_i^{(t)})P(x_i^{(t)})}{P(y_t)}$. So in principle, we could estimate the probability of each trajectory, and then pick the trajectory with the highest probability.

The problem with this simple approach is that it is very wasteful. Because the probabilities multiply, all it takes is one low probability measurement to make the entire trajectory very unlikely. But what if instead, we could adaptively allocate more compute to the trajectories that look more promising? This is the idea behind sequential monte carlo. To do this, what we do is that after every step, we take our set of particles $\hat{X}^{(t+1)} = \{x_1^{(t+1)}, x_2^{(t+1)}, ..., x_N^{(t+1)}\}$ and we resample them with replacement, where the probability of sampling each particle is proportional to the probability of that particle given the measurement.

So a simple version of the algorithm is as follows:

- Start with an initial distribution $P_0(x)$ from which we are able to sample.
- Independently sample set of particles $X^{(0)} = \{x_1^{(0)}, x_2^{(0)}, ..., x_N^{(0)}\}$ from P_0.
- Sample a set of particles $\hat{X}^{(t+1)} = \{x_1^{(t+1)}, x_2^{(t+1)}, ..., x_N^{(t+1)}\}$ from $P(x^{(t+1)} \mid x^{(t)})$
- Evaluate the importance weights $W^{(t+1)} = \{w_1^{(t+1)}, w_2^{(t+1)}, ..., w_N^{(t+1)}\}$ for the samples in $\hat{X}^{(t+1)}$; intuitively, these weights measure how "promising" each sample is to evolve into a representative sample from the target distribution π. In this case, the weights can be computed as $w_i^{(t+1)} = P(y_{t+1}|x_i^{(t+1)})$.
- Resample the particles in $\hat{X}^{(t+1)}$ with replacement according to their importance weights $W^{(t+1)}$ to obtain the next set of particles $X^{(t+1)}$.

This algorithm will give you an estimate of the probability distribution $P(x_{0:t}|y_1, y_2, ..., y_t)$. Note that $P(x_t|y_{0:t})$ is different from the distribution $P(x_t|y_t)$. In particular, if $P(x_t|y_t)$ has multiple modes of different sizes, then

$P(x_t|y_1, y_2, ..., y_t)$ will tend to focus on one of the modes, usually the largest one, because of the accumulated evidence from the previous measurements. This general approach is useful in many contexts; famously, the Schmidt-Kalman filter, which is a specific instance of SMC, was used by NASA to track the position of the Apollo 11 spacecraft on its way to the moon.

Now, what does this have to do with synthesis? Recall that we have already established that π is a stationary distribution of a markov process K. So we can think of the markov process as describing how a particle moves over time, and the observation is simply that we have observed that the particle is in the stationary distribution, which implies that $P(x_t|y_t) = \pi(x_t)$.

This means that if we start with a random distribution of particles $X^{(0)} = \{x_1^{(0)}, x_2^{(0)}, ..., x_N^{(0)}\}$ sampled from some initial distribution $J_0(x)$, and we evolve the particles according to K, and at each step we resample the particles according to $\pi(x_t)$, then over time, the particles will converge to the mode of the stationary distribution π (not to the distribution π itself, but to its mode).

It is possible to make adjustments to the algorithm to make it converge to the distribution π itself, but that is not necessarily more useful if we just want to find a good program.

We will revisit this algorithm again in the next unit, where we will see how it can be used to guide a large language model to synthesize programs that satisfy a specification.

References

[1] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS '13, Houston, TX, USA - March 16 - 20, 2013*, pages 305–316, 2013. doi: 10.1145/2451116.2451150. URL http://doi.acm.org/10.1145/2451116.2451150.

[2] Persi Diaconis. The markov chain monte carlo revolution. *Bulletin of the American Mathematical Society*, 46:179–205, 2009. URL https://doi.org/10.1090/S0273-0979-08-01238-X.

[3] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. URL https://doi.org/10.1063/1.1699114.

[4] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design, FMCAD 2013, Portland, OR, USA, October 20-23, 2013*, pages 1–8, 2013. URL http://ieeexplore.ieee.org/document/6679385/.

[5] Arnaud Doucet, Nando de Freitas, and Neil Gordon. *An Introduction to Sequential Monte Carlo Methods*, pages 3–14. Springer New York, New York, NY, 2001. ISBN 978-1-4757-3437-9. doi: 10.1007/978-1-4757-3437-9_1. URL https://doi.org/10.1007/978-1-4757-3437-9_1.